

2026-05-19-const046-i18n-architecture-design

Round 90 — CONST-046 i18n Architecture Design

Date: 2026-05-19 **Status:** Design Proposal (pending operator decision points in §10)
Programme: HelixCode §11.4 governance campaign, round 90 — documentation-only **Scope:** Top-level i18n architecture for HelixCode and all owned submodules carrying deferred CONST-046 backlog **Constitutional anchors:** CONST-035 (anti-bluff), CONST-044 (CONTINUATION discipline), CONST-046 (no-hardcoded-content), CONST-047 (recursive submodule application), CONST-049 §11.4.17 (universal-vs-project classification), CONST-050(A) (no-fakes-beyond-unit-tests), CONST-051(B) (decoupling), CONST-051(C) (dependency layout), Article XI §11.9 (end-user usability guarantee)

1. Background

1.1 The CONST-046 Mandate (verbatim, from this repo's CLAUDE.md §10.5)

“NO user-facing text, question template, prompt text, error message, label, helper text, or explanatory content may be hardcoded as a static literal string in any source file. All text visible to users MUST be: 1. Generated dynamically by an LLM at runtime based on the user's language, prompt context, and session state, OR 2. Loaded from an i18n resource file (.yaml, .json, .toml) with locale-aware overrides, OR 3. Composed programmatically from verifier metadata, provider responses, or configuration data.”

Why it matters (verbatim from §10.5): *“Hardcoded English strings silently break the product for non-English users. A clarification question hardcoded as ‘Which file has the bug?’ is asked identically to Serbian, Japanese, or Spanish users — producing an incoherent, unusable experience.”* This is a §11.9 / Article XI end-user-usability defect of the same severity class as a PASS-bluff.

1.2 Operator anti-bluff anchor (2026-05-19)

“all existing tests and Challenges do work in anti-bluff manner - they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!”

The architecture below is constrained by this anchor: any i18n abstraction whose tests pass on a missing key without surfacing the gap is, itself, a §11.4 PASS-bluff regardless of how green its summary line is. §6 covers the loud-failure design that protects against this.

1.3 Accumulated CONST-046 backlog (round-by-round deferrals)

Cross-referenced from docs/CONTINUATION.md close-outs:

Submodule	File(s)	Hits	Deferral round	Notes
HelixLLM	internal/agents/ tools/web.go, internal/gateway/ openai.go	2	rounds 23-29 audit	Hardcoded English in tool prompts + gateway error messages
Planning (hiplan)	mcts/ TreeOfThoughts.go and siblings	3+	rounds 23-29 audit	Prompt templates for tree-of- thoughts reasoning
SelfImprove	reward + integration prompts	8	rounds 23-29 audit	LLM reward- modelling prompts, all English
VisionEngine	StubAnalyzer paths	4	rounds 23-29 audit	Analyzer status strings
panoptic	multiple	unspecified	round 29	Listed as “CONST-046 deferral”
DocProcessor	cmd/docprocessor/ main.go lines 35, 44, 46, 50, 53	5	round 29 / re- confirmed round 73 (close-out ¹¹⁶)	CLI status output
harmony_os	various	unspecified	round 31 (one admission removed; siblings remain)	Platform- feature messages

Conservative estimate: ~22-30 string occurrences across 7 submodules. None are §11.4 PASS-bluffs in the strict sense (no fabricated runtime evidence); all are end-user-usability defects per §11.9.

1.4 Why this is round 90 and not round 91+ implementation

The backlog has been deferred 50+ rounds because every individual hit is small but the **abstraction is project-wide**: each submodule that adopts an ad-hoc i18n shim creates exactly the kind of coupling CONST-051(B) forbids. A single shared design — designed once, applied everywhere — is the only CONST-047-compliant resolution. Round 90 produces that design. Rounds 91+ execute it.

2. Constraints and goals

2.1 Hard constraints

- **CONST-051(B) decoupling:** no owned submodule may import a HelixCode-specific package. Every submodule consuming `i18n` MUST receive its `Translator` via dependency injection from the parent application's composition root.
- **CONST-050(A) production purity:** `i18n` implementation MAY use mocks only in unit tests. Integration tests load real bundle files from disk.
- **CONST-046 loud-failure invariant:** a missing key MUST NOT silently render as empty string, “(missing)” placeholder, or fabricated text. Missing key → either return key literal as fallback (English source-key acts as graceful degradation) OR surface `ErrI18nKeyMissing` sentinel. Choice is per-call via API.
- **CONST-053 .gitignore:** no generated/compiled bundle artefacts in version control. Source YAML files only; any pre-compilation step must regenerate from source.
- **Pure Go preferred:** minimise external dep weight; avoid CGo.
- **OS-level locale detection:** respect `LC_ALL`, `LANG` environment variables per POSIX; allow per-request override.

2.2 Soft goals

- Bundle files human-editable (translators are not always programmers).
- Pluralisation support (English `1 file` vs `2 files`; Slavic forms more complex — Serbian has 3-form plural).
- Variable interpolation with type safety.
- Lazy loading (don't read every locale's full bundle on every process start).
- Translator API surface small enough to fit in one page of Go.

3. Architecture options

Four options surveyed. Each evaluated on: dep weight, locale-resolution semantics, fallback behaviour, pluralisation, decoupling fit, ease of editing, alignment with existing Go ecosystem usage in HelixCode tree.

3.1 Option A — YAML resource bundles (in-house loader)

Description: One YAML file per locale (e.g., `i18n/en.yaml`, `i18n/sr.yaml`). Custom Go loader using `gopkg.in/yaml.v3`. Simple `map[string]string` keyed by dot-namespaced ID (e.g., `docprocessor.cli.loaded`).

Pros: - Zero new external deps beyond YAML lib (already transitively present). - Trivially auditable: bundle files are plain text, diffs reviewable in PRs. - Pure Go, no CGo. - Loader can be ~150 LOC; trivial to mutation-test.

Cons: - Pluralisation requires custom CLDR-plural-rule logic (handwritten = error-prone for Slavic three-form plurals). - Interpolation is `text/template`-based; loses type safety vs typed-generator approaches. - No tooling for translator hand-off (no `xgettext`-style extractor; we'd write our own).

Locale resolution: parse `LC_ALL/LANG` → BCP-47 → strip region (`sr_RS.UTF-8` → `sr-RS` → `sr`). Per-request override accepted.

Fallback: requested → language-only (sr-RS → sr) → en-US (default) → key literal OR sentinel error (caller chooses).

3.2 Option B — Go-struct embedded resources (embed.FS + text/template)

Description: Bundle files committed under `i18n/`, embedded into the binary via `//go:embed i18n/*.yaml`. Same YAML format as Option A; the difference is distribution — bundles travel inside the binary, no runtime file I/O.

Pros: - Zero runtime filesystem dependency (single-binary deploys clean). - All Option A pros otherwise.

Cons: - Every locale ships in every binary even when unused (binary bloat — small per-locale, but multiplies). - Translation updates require recompile + redeploy (no live patch). - All Option A cons otherwise. - Conflicts with CONST-051(B) decoupling when used inside a submodule — the submodule's binary embeds bundles the consumer might want to override.

Locale resolution / fallback: identical to Option A.

3.3 Option C — gettext-style .po files with golang.org/x/text/message

Description: Use Google's official Go i18n package (`golang.org/x/text/message`, `language`, `catalog`). Bundles stored as `.po` or generated catalogs.

Pros: - Tier-1 maintained by the Go team; transitively present in HelixCode tree (`panoptic`, `helix_qa`, many `cli_agents/*`). - Strong CLDR-grounded pluralisation built in. - `gotext` tool extracts strings and generates skeleton catalogs.

Cons: - API designed for **gettext** workflow (catalogs compiled from `.po` → Go source). Compiled catalogs are generated code — conflicts with CONST-053 if checked in; if regenerated at build, adds a build step every submodule must wire up. - `.po` format is less ergonomic than YAML for non-translator editors. - Catalog struct-tag mechanism encourages compile-time binding (works against the dependency-injection pattern CONST-051(B) requires).

Locale resolution: `language.MatchStrings` with operator-supplied matcher. Excellent CLDR fidelity.

Fallback: explicit `Matcher` configured at construction; `Fallback()` chain user-defined.

3.4 Option D — nicksnyder/go-i18n/v2 (JSON/YAML bundles)

Description: Popular third-party Go i18n library. Bundles as JSON, YAML, or TOML. Already transitively present in `cli_agents/crush/` and in the `kimi_agent_helix_cli_integration_blueprint` reference materials.

Pros: - Ergonomic API: `Localizer.Localize(&i18n.LocalizeConfig{MessageID: "..."})`. - CLDR pluralisation built in (uses Google's CLDR data tables). - YAML bundle support out of the box (translator-friendly). - Already battle-tested across the Go ecosystem. - One external dep with permissive MIT licence; transitively already pulled in by `cli_agents/crush/`.

Cons: - External dep — version-pinning + audit burden (small but real per CONST-042 supply-chain hygiene). - Library has its own opinions about pluralisation key suffixes (`.one`, `.other`, `.few`, `.many`) — translators must learn its conventions. - Default behaviour on missing key returns `MessageID` (the key itself) — matches our loud-failure preference but must be explicitly chosen via `LocalizeConfig.DefaultMessage`.

Locale resolution: `language.Tag` parsed from `Accept-Language` header or env var; `Bundle.MatchTags` does the fallback walk.

Fallback: requested → bundle's `MatchTags` → default tag → message ID literal (our loud-failure path).

4. Recommendation

Option D — `nicksnyder/go-i18n/v2` is recommended.

Rationale:

1. **Ecosystem alignment** — already transitively in tree (`cli_agents/crush/go.mod:150`) and explicitly chosen by the `kimi_agent_helix_cli_integration_blueprint` reference research. Re-using what the Go ecosystem has converged on is cheaper than rolling our own (Option A) or fighting `gotext`'s codegen workflow (Option C).
2. **YAML bundle support** — keeps the human-editability advantage of Option A without writing our own loader.
3. **CLDR pluralisation** — Serbian three-form plurals (`1 fajl, 2 fajla, 5 fajlova`) work correctly without us hand-coding plural rules. Hand-coding plural rules is a §11.4.6 (no-guessing) risk magnet.
4. **Loud-failure compatible** — missing key returns the message ID literal by default; we tighten to sentinel via wrapper.
5. **Decoupling fit** — the library exposes `*i18n.Bundle` and `*i18n.Localizer` as pure types with no global state. Submodules consume our `Translator` interface (defined in §5.1); the concrete `go-i18n`-backed implementation lives only in the parent app's composition root. Submodules never import `"github.com/nicksnyder/go-i18n/v2/i18n"` directly — they import our interface. CONST-051(B) satisfied.

What the recommendation does NOT pick: - **Not Option B (embed.FS):** binary-bloat + recompile-to-update tradeoff dominates. - **Not Option A (in-house):** hand-coding plural rules is a §11.4.6 risk; we'd be re-implementing CLDR plural tables badly. - **Not Option C (x/text/message):** codegen workflow conflicts with our YAML-bundles-as-source-of-truth preference; catalog struct-tag binding fights DI.

Honest tradeoff acknowledgement: Option A's "zero external deps" is genuinely attractive under CONST-042. The recommendation accepts one MIT-licensed dep with permissive audit history; rounds 91-93 MUST include the dep-audit step explicitly.

5. Components

5.1 Top-level: helix_code/pkg/i18n/

New top-level package (exported, hence `pkg/` not `internal/`) defining the consumer-facing interface every submodule depends on:

```
package i18n

import "context"

// Translator is the contract every submodule depends on for
// user-facing text.
// Implementations are injected from the parent application's
// composition root.
type Translator interface {
    // T resolves a key for the request's locale, interpolating
    // named args.
    // Returns the resolved string. On missing key, returns key
    // literal as fallback.
    T(ctx context.Context, key string, args map[string]any)
        string

    // TStrict is identical to T but returns ErrI18nKeyMissing on
    // missing key.
    TStrict(ctx context.Context, key string, args
        map[string]any) (string, error)

    // TN handles pluralisation (CLDR rules). count drives plural
    // form.
    TN(ctx context.Context, key string, count int, args
        map[string]any) string

    // WithLocale returns a Translator bound to a specific BCP-47
    // tag,
    // bypassing context-based locale extraction.
    WithLocale(tag string) Translator
}

var ErrI18nKeyMissing = errors.New("i18n: key not found in any
    locale bundle")
```

Concrete implementation `pkg/i18n/gobundle/` wraps `nicksnyder/go-i18n/v2`. Tests live alongside.

5.2 Per-submodule consumption pattern

Every owned submodule that needs user-facing text:

1. Declares a constructor parameter of type `i18n.Translator` (the interface above, vendored or accessed via Go module path — TBD in §10).
2. Stores it on the receiver struct.
3. Calls `r.t.T(ctx, "submodule.feature.key", args)` instead of `fmt.Sprintf("English literal: %s", v)`.

Example — DocProcessor CLI migration:

```
// Before (CONST-046 violation):
fmt.Printf("Loaded %d documents\n", len(docs))

// After:
fmt.Println(r.t.T(ctx, "docprocessor.cli.loaded",
    map[string]any{"Count": len(docs)}))
```

Bundle entry (submodules/doc_processor/i18n/en.yaml):

```
docprocessor:
  cli:
    loaded: "Loaded {{.Count}} documents"
```

Serbian counterpart (i18n/sr.yaml):

```
docprocessor:
  cli:
    loaded:
      one: "Učitan {{.Count}} dokument"
      few: "Učitana {{.Count}} dokumenta"
      other: "Učitano {{.Count}} dokumenata"
```

5.3 Bundle storage layout

Per CONST-051(C) (dependency layout) and CONST-051(B) (decoupling), each submodule keeps its own bundles at its own repo root:

```
<submodule>/
  i18n/
    en.yaml      # required, source-of-truth locale
    sr.yaml      # optional additional locales
    ja.yaml
    ...
```

The submodule MUST NOT ship a `Translator` implementation — only bundle files. The parent application (`HelixCode`) constructs the `Translator` from the union of all submodules' bundles at startup. Construction is wired in `helix_code/cmd/server/main.go` (and equivalent CLI / desktop / mobile entry points).

5.4 Default locale and fallback chain

- **Default:** en-US (chosen because it is the source-of-truth locale every key MUST appear in; matches existing English literals).
- **Fallback walk:** requested tag → language-only stripped (sr-RS → sr) → en-US → loud failure (key literal or `ErrI18nKeyMissing`).
- **NEVER fabricate:** no machine-translated synthetic text at runtime. If a translator wants to add a locale, they add the YAML file. Absence of locale-N for a key is a graceful en-US fallback, never a hallucination.

5.5 Locale source priority (highest wins)

1. Per-call `WithLocale(tag)` override.
 2. `context.Context` value (`ctx.Value(i18n.LocaleKey)`).
 3. Process env `LC_ALL`.
 4. Process env `LANG`.
 5. Hardcoded fallback en-US.
-

6. Anti-bluff guarantee

The §1.2 operator anchor forbids tests that pass without proving end-user functionality. Three concrete protections:

1. **Loud missing-key:** as §5.4 — fallback is key literal (visible to the user as a degraded experience they can report) or sentinel error (caller's choice). No silent empty string, no fabricated placeholder.
 2. **Paired mutation tests** (CONST-050(A) / §11.4 paired mutation): every i18n migration ships with TWO tests:
 - **Coverage test:** asserts the key resolves for en-US and at least one non-English locale.
 - **Mutation test:** deletes the key from en-US bundle, asserts the strict path returns `ErrI18nKeyMissing` AND the lenient path returns the key literal. Test FAILs if either condition silently produces a non-empty translation that wasn't the literal — proving the loud-failure machinery actually fires.
 3. **CONST-046 audit gate:** round 91 ships `scripts/audit-const046.sh` that greps for `fmt.Printf/fmt.Println/fmt.Errorf/return-string-literal` patterns inside production paths (`cmd/`, `internal/`, `applications/`, `pkg/`) of every owned submodule. Output: hit list with file:line. Round 91+ migrations clear the list; the gate runs in `make ci-validate-all` warn-mode initially and tightens to fail-on-hit once Phase 3 completes.
-

7. Phased implementation plan

Each phase is intended to be 2-3 rounds of subagent-driven-development. Round counts are estimates; operator may compress or expand.

Phase 1 — Foundation (rounds 91-93)

- **Round 91:** Create `helix_code/pkg/i18n/` package with `Translator` interface + `gobundle/` concrete impl wrapping `nicksnyder/go-i18n/v2`. Unit tests + paired

mutation tests. Wire into `helix_code/cmd/server/main.go` composition root. Ship `helix_code/i18n/en.yaml` skeleton.

- **Round 92:** Ship `scripts/audit-const046.sh` in warn-mode. Run it; populate `docs/const046-audit-baseline.md` with full hit list (single source of truth for what Phase 2/3 has to clear).
- **Round 93:** Add Translator-injection wiring to the 7 backlog submodules (no string migration yet — only the injection surface). Each submodule’s CLAUDE.md gains an “i18n” sub-section documenting the `Translator` parameter and bundle-file location.

Phase 2 — Top-impact migrations (rounds 94-96)

Migrations are ordered by user-visibility × hit-count:

- **Round 94:** `SelfImprove` × 8 prompts. Highest hit-count single submodule; LLM-facing prompts so locale-aware prompting may improve non-English reward quality.
- **Round 95:** `HelixLLM` × 2 strings (`web.go` tool prompt + `openai.go` gateway error). Customer-facing.
- **Round 96:** `harmony_os` string sweep. Platform-feature strings visible in user-facing error paths.

Each round: migrate, ship paired mutation tests, ship `sr` and `ja` locales for the migrated keys (operator may choose alternate seed locales in §10).

Phase 3 — Remainder (rounds 97-99)

- **Round 97:** `DocProcessor CLI` (`cmd/docprocessor/main.go 5 fmt.Printf` lines).
- **Round 98:** `Planning hiplan / MCTS / TreeOfThoughts` × 3+ prompt templates.
- **Round 99:** `VisionEngine` × 4 `StubAnalyzer` strings, panoptic remainder. Tighten `scripts/audit-const046.sh` from warn-mode to fail-on-hit.

Phase exit criteria

A phase is closed when: - All migrations of the phase ship with paired mutation tests AND captured runtime evidence per §11.4.2 (terminal output pasted in close-out). - `scripts/audit-const046.sh` hit count strictly decreases vs prior phase’s baseline. - CONTINUATION close-out documents the phase-end audit count + per-submodule status.

8. Test strategy

Per CONST-050(B), four-layer test floor for every migration:

Layer	What it proves	Example for <code>DocProcessor cli.loaded</code>
Pre-build static	Bundle YAML files parse + every key in <code>en.yaml</code> is present in declared-required locales	<code>go test ./pkg/i18n/audit/</code>
Post-build unit	<code>Translator.T</code> resolves the key for en-US and falls	<code>TestTranslator_DocProcessorCLILoaded_EN</code>

Layer	What it proves	Example for DocProcessor cli.loaded
	back correctly for missing locale	
Runtime integration	Real CLI run with <code>LANG=sr_RS.UTF-8</code> ./docprocessor prints Serbian text; same with <code>LANG=en_US.UTF-8</code> prints English	<code>tests/integration/i18n/docprocessor_test.go</code>
Paired mutation	Delete the key from <code>en.yaml</code> ; strict path returns <code>ErrI18nKeyMissing</code> , lenient returns key literal	<code>TestTranslator_DocProcessorCLILoaded_Missing</code>

Captured-evidence requirement: integration tests MUST capture actual stdout — diff against expected per CONST-035 / §11.4.2.

9. Cross-references

- **CONTINUATION close-out**¹¹⁶ (round 73): DocProcessor CLI 5 `fmt.Printf` lines — explicitly deferred and routed to this design.
- **CONTINUATION line 132** (P3-T04): clarification engine LLM-driven per CONST-046 — option (1) of the CONST-046 mandate (LLM-generation). Demonstrates the LLM path that complements this YAML-bundle path; both options are needed for different use cases.
- **CONTINUATION line 140**: CONST-046 mandate adoption.
- **Round 31** (`harmony_os`): one “simulated results” admission removed; sibling strings flagged for this design.
- **Round 37 spec** (`2026-05-18-round37-memory-persistence-real-backends-design.md`): demonstrates the injection-point pattern (`nil-client sentinel` → `loud-failure`) that §6’s loud-failure design reuses.
- **CONST-046 in CLAUDE.md §10.5** of this repo: the mandate text + compliance examples.
- **Constitution submodule** (`constitution/Constitution.md §11.4.x`): the universal severity classification.

10. Operator decision points

These resolve before round 91 implementation starts. Each has a recommended default; operator may override.

1. **Module path for the Translator interface**: should `pkg/i18n/` live in HelixCode and be vendored into each submodule, OR be promoted to a dedicated own-org submodule (`vasic-digital/HelixI18n` or similar) per CONST-051(C) flat-layout discipline?
Recommended: dedicated submodule — keeps decoupling clean and allows non-HelixCode consumers (e.g., panoptic standalone use) to depend on it without pulling HelixCode. Round 91 first task = create the submodule.
2. **Seed non-English locales**: which locale(s) ship alongside `en-US` for Phase 2?
Recommended: `sr-RS` (operator’s locale per repo signals) + `ja-JP` (high-divergence script — good fallback-walk test). Operator may add/swap.

3. **Audit gate tightening cadence:** when does `scripts/audit-const046.sh` flip from warn-mode to fail-on-hit? **Recommended:** end of Phase 3 (round 99). Earlier flip risks blocking unrelated work; later flip risks regression-creep.
 4. **LLM-generation vs i18n-bundle split:** the CONST-046 mandate allows both.
Recommended split: clarification questions + tool prompts + LLM-internal prompts → LLM-generation (option 1, locale propagated to the LLM via system prompt); CLI status output + error messages + UI labels + helper text → i18n bundles (option 2, this design). The clarification engine in `internal/clarification/` is the reference for option 1.
 5. **Pluralisation locale subset:** ship pluralised entries for `en` (one/other) and `sr` (one/few/other) from Phase 2 onward, OR defer pluralisation entirely to a later phase?
Recommended: ship from Phase 2 — `go-i18n/v2` handles it for free, and retro-fitting plural forms after the fact is more painful than designing them in.
-

11. Self-review checklist (per superpowers:brainstorming)



No placeholders / TODOs in the spec body



Internal consistency: §3 options ☐ §5 components ☐ §7 phases all reference the same Translator interface and YAML bundle layout



Scope check: 3 phases × ~3 rounds = round-sized work each



Ambiguity: every option's pros/cons stated, recommendation explicitly chosen and justified, honest tradeoff acknowledgement included (§4 last paragraph)



Anti-bluff: §6 loud-failure design + §8 paired mutation requirement enforced per migration



Constitutional alignment: CONST-046, CONST-051(B/C), CONST-050(A), CONST-047 all explicitly addressed



Operator decisions enumerated in §10 — none silently assumed



Word count target (~3000) and LOC budget (<600 markdown lines) respected